

Formal Review Request Template

Requester: __Vedaant
Patel_____

Reviewer: _____Alvin
Mawlue_____

Date:
_____10/6/2026_____

Approx. review duration: ____45
mins____

Choose **one primary review method**:

Code / Software Review

- ☒ **Code Quality Review**
Inspect the code for issues such as readability, modularity, maintainability, efficiency, and appropriate programming practices.
- ☐ **White-Box / Edge-Case Testing**
Inspect the internal logic and systematically test branches, boundary conditions, exceptions, or adversarial inputs.
- ☐ **Black-Box / Functional Testing**
Develop test cases from the expected requirements or behavior and determine whether the software produces the expected results.
- ☐ **Performance / Runtime Testing**
Systematically evaluate whether the software meets relevant runtime or performance expectations.

Usability Review

- ☐ **Task-Based Usability Testing**
Give the reviewer/user specific tasks to complete while you observe what they do, where they succeed, and where they encounter problems. A think-aloud protocol may be used.

☐ Heuristic / Guidelines Evaluation

Systematically inspect the interface using identified usability principles, heuristics, or design guidelines.

- ☐ **Prototype Usability Testing**
Have users complete defined tasks using a wireframe, prototype, or partially implemented interface.

Design Review

- ☐ **Architecture / Conceptual Design Review**
Evaluate whether architecture and major design decisions align with project requirements and constraints.
- ☐ **Preliminary Design / UML Review**
Evaluate proposed system structure, diagrams, component relationships, interfaces, or other design artifacts.
- ☐ **Security Design Review**
Evaluate relevant authentication flows, trust boundaries, data protection, error handling, or other security-related design decisions.

Other

- ☐ **Other review method:**

Explain what the reviewer will evaluate and why this method is appropriate for your work:

Review Request Details

1) What specifically should the reviewer examine or interact with?

Certain list of code files that I worked on this sprint for my specified features. Focus on readability, maintainability, and proper programming practices.

2) What question do you want this review to help answer?

Questions:

- Is the code readable and fairly easy to understand
 - Is the code easy to maintain and modify
 - Is the code organized and structured appropriately
 - Are there any areas that could be simplified or improved
-

3) What should the reviewer focus on?

- Readability and clarity of the code
 - Maintainability and organization
 - Proper Ruby/Rails programming practices
 - Avoiding unnecessary or duplicated code
 - Appropriate use of methods, classes, and existing functionality
 - Potential areas for refactoring or improvement
-

4) What should the reviewer *not* evaluate during this review?

- Do not evaluate the visual design or UI polish.
 - Do not evaluate features or code outside of the provided files.
 - Do not evaluate unfinished features that were not part of this sprint.
-

5) What should the reviewer use to judge the work?

The reviewer should use the project's existing requirements and expected behavior, along with standard Ruby/Rails coding best practices. The code should be judged based on whether it is clear, maintainable, appropriately structured, and follows reasonable programming practices.

Reviewer Section

1) Briefly describe what you actually did during the review.

Examples include executing defined test cases, inspecting particular code paths, observing a user completing defined tasks, or systematically examining a design against specified criteria.

I review `app/models/ability.rb`, `app/controllers/students_controller.rb`, and `app/controllers/teams_controller.rb` for readability, maintainability

2) What portions of the artifact did you evaluate?

3) Provide or link to relevant evidence such as test cases, screenshots, notes, annotated code, observations, or other review artifacts.

StudentsController [fall2026_tagem_up/app/controllers/students_controller.rb at main · uofm-capstone/fall2026_tagem_up](#)

TeamsController [fall2026_tagem_up/app/controllers/teams_controller.rb at main · uofm-capstone/fall2026_tagem_up](#)

Findings

Strengths:

- _____
- _____

Issues / Defects:

Description of Issue	Severity (Low/Med/High)	Evidence / Example	Suggestions
Authorization bypassed	High	The <code>rescue nil</code> on <code>load_and_authorize_resource</code> and both <code>authorize!</code> calls swallows	Remove the <code>rescue nil</code> modifiers and handle denials with <code>rescue_from</code> in

		CanCan::AccessDenied, so unauthorized users can create and edit students	ApplicationController
team_params	Med	team_params permits student_ids: [], so a crafted request can attach any student, from any semester, to a team	Either drop <code>student_ids</code> from the permitted params and use <code>add_member</code> , or validate the IDs against the current semester or leave it as it is. Maybe we need to access previous semester records

Requester Follow-Up

1) How will you address the feedback? If you decide **not** to, briefly explain why.

- I will take the feedback and try to adapt to the best of my abilities. I value outside opinions so I will not ignore the feedback.
